

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.
4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.
5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

Reg.No : 192524317

Name : Dhanush S

1. Star Topology

This program accepts the number of hosts and displays the hosts connected to a central switch with Cat6 cable lengths.

Python Code:

```
# Star Topology

n = int(input("Enter number of hosts: "))

switch = "Central Switch"

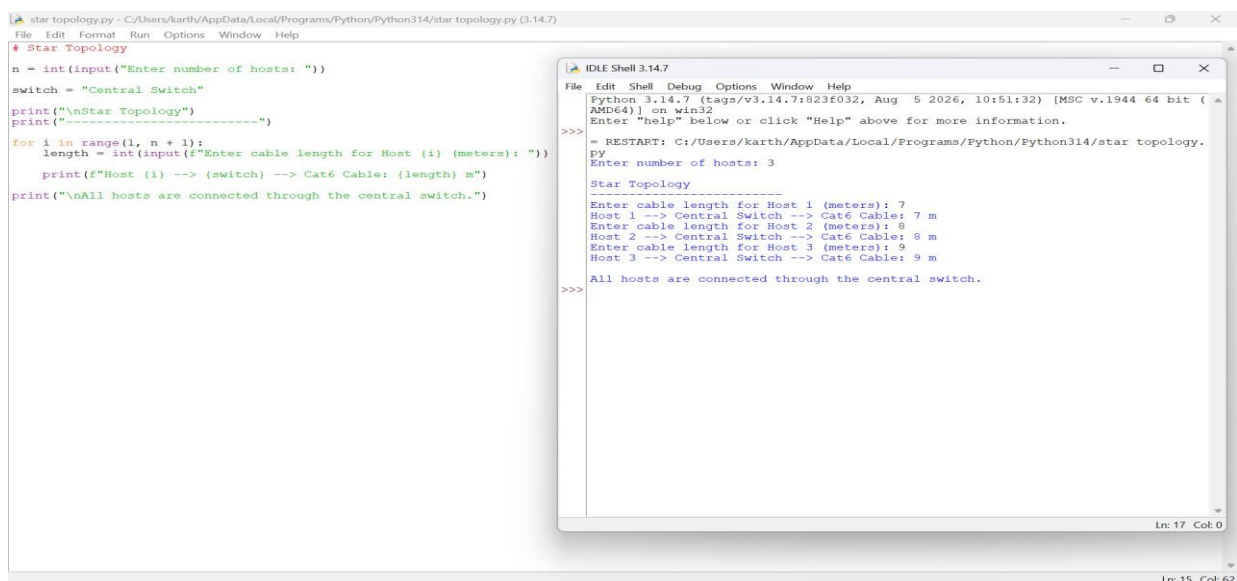
print("\nStar Topology")
print("-----")

for i in range(1, n + 1):
    length = int(input(f"Enter cable length for Host {i} (meters): "))

    print(f"Host {i} --> {switch} --> Cat6 Cable: {length} m")

print("\nAll hosts are connected through the central switch.")
```

Output Screenshot:



```
star topology.py - C:/Users/karth/AppData/Local/Programs/Python/Python314/star topology.py (3.14.7)
File Edit Format Run Options Window Help
# Star Topology
n = int(input("Enter number of hosts: "))
switch = "Central Switch"
print("\nStar Topology")
print("-----")
for i in range(1, n + 1):
    length = int(input(f"Enter cable length for Host {i} (meters): "))
    print(f"Host {i} --> {switch} --> Cat6 Cable: {length} m")
print("\nAll hosts are connected through the central switch.")

IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:/Users/karth/AppData/Local/Programs/Python/Python314/star topology.py
PY
Enter number of hosts: 3
Star Topology
-----
Enter cable length for Host 1 (meters): 7
Host 1 --> Central Switch --> Cat6 Cable: 7 m
Enter cable length for Host 2 (meters): 8
Host 2 --> Central Switch --> Cat6 Cable: 8 m
Enter cable length for Host 3 (meters): 9
Host 3 --> Central Switch --> Cat6 Cable: 9 m
>>> All hosts are connected through the central switch.
>>>
```

Concept: In a star topology, every host is directly connected to a central switch.

2. UTP Cable Length Validation:

The question requires checking whether a UTP cable is within the 100-meter maximum.

Python code:

```
# UTP Cable Length Validation
```

```
def validate_segment(length_m):  
    if length_m <= 100:  
        return True, "Valid cable length"  
    else:  
        return False, "Error: Cable length exceeds 100 meters"
```

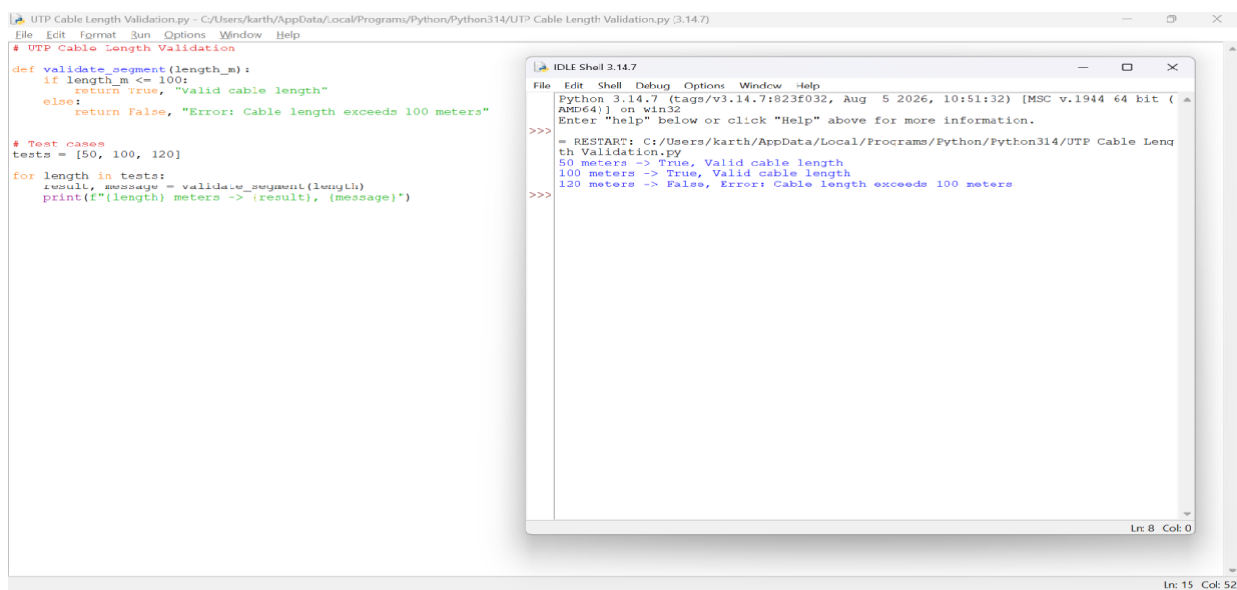
```
# Test cases
```

```
tests = [50, 100, 120]
```

```
for length in tests:
```

```
    result, message = validate_segment(length)  
    print(f'{length} meters -> {result}, {message}')
```

Output Screenshot:



Simple Interpretation:

- 50 m: Valid
- 100 m: Valid
- 120 m: Invalid because it exceeds the 100 m limit

3. CSMA/CD Simulation:

The PDF asks for four hosts, collision detection, random back-off, transmission attempts, and logging in csma_log.txt.

Python Code:

```
# Simple CSMA/CD Simulation

import random
import time

hosts = ["Host1", "Host2", "Host3", "Host4"]

log = open("csma_log.txt", "w")

for host in hosts:

    attempt = 1

    while True:
        log.write(f'{host}: Transmission attempt {attempt}\n')

        # Random collision
        collision = random.choice([True, False])

        if collision:
            log.write(f'{host}: Collision detected\n')

            backoff = random.randint(1, 3)
            log.write(f'{host}: Back-off = {backoff} seconds\n')

            print(host, "Collision! Back-off:", backoff, "seconds")
            time.sleep(backoff)

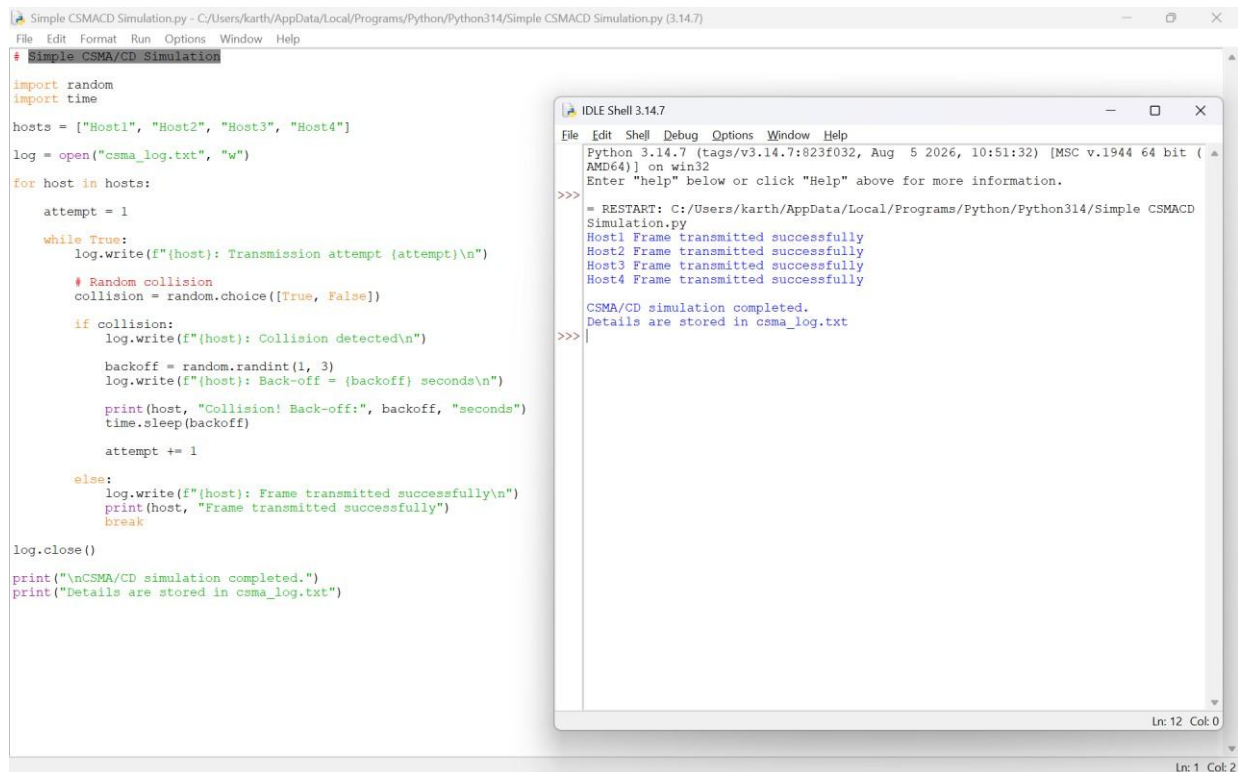
            attempt += 1

        else:
            log.write(f'{host}: Frame transmitted successfully\n')
            print(host, "Frame transmitted successfully")
            break

log.close()

print("\nCSMA/CD simulation completed.")
print("Details are stored in csma_log.txt")
```

Output Screenshot:



The screenshot displays a Python IDE window titled 'Simple CSMA/CD Simulation.py' and an 'IDLE Shell 3.14.7' window. The script in the IDE simulates a CSMA/CD protocol for four hosts. The shell window shows the execution output, indicating successful frame transmissions for all hosts and the completion of the simulation.

```
# Simple CSMA/CD Simulation

import random
import time

hosts = ["Host1", "Host2", "Host3", "Host4"]
log = open("csma_log.txt", "w")

for host in hosts:
    attempt = 1
    while True:
        log.write(f"{host}: Transmission attempt {attempt}\n")
        # Random collision
        collision = random.choice([True, False])

        if collision:
            log.write(f"{host}: Collision detected\n")
            backoff = random.randint(1, 3)
            log.write(f"{host}: Back-off = {backoff} seconds\n")
            print(host, "Collision! Back-off:", backoff, "seconds")
            time.sleep(backoff)
            attempt += 1
        else:
            log.write(f"{host}: Frame transmitted successfully\n")
            print(host, "Frame transmitted successfully")
            break
    log.close()

print("\nCSMA/CD simulation completed.")
print("Details are stored in csma_log.txt")
```

```
>>>
= RESTART: C:/Users/karth/AppData/Local/Programs/Python/Python314/Simple CSMA/CD
Simulation.py
Host1 Frame transmitted successfully
Host2 Frame transmitted successfully
Host3 Frame transmitted successfully
Host4 Frame transmitted successfully

CSMA/CD simulation completed.
Details are stored in csma_log.txt
>>>
```

4. Client-Server Socket Program:

Python Code:

```
import socket
import threading
import time

def server():
    s = socket.socket()
    s.bind(("localhost", 5000))
    s.listen(1)
    print("Waiting for client...")

    c, addr = s.accept()
    print("Client connected")

    msg = c.recv(1024).decode()
    print("Frame received:", msg)

    print("\nMAC Address Table:")
    print("AA:BB:CC:DD:01 -> Port 1")
    print("AA:BB:CC:DD:02 -> Port 2")
```

```
c.send("Frame received".encode())
c.close()
s.close()
```

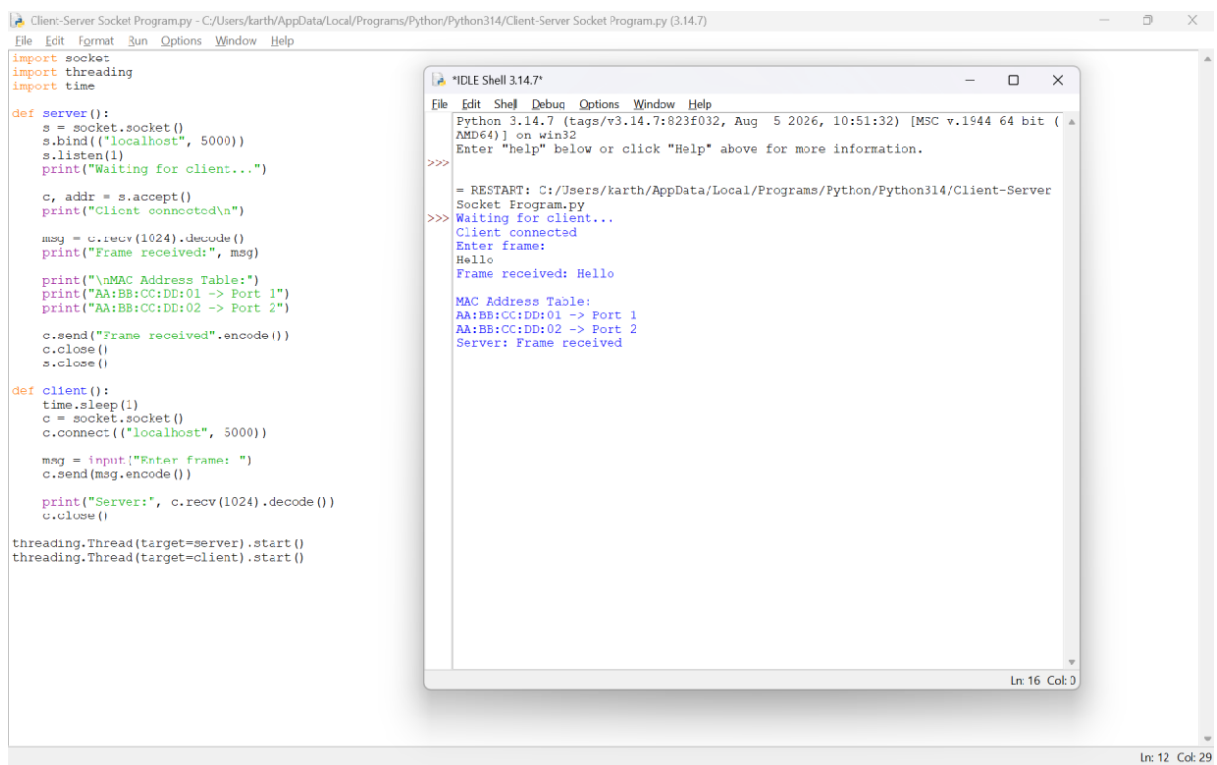
```
def client():
    time.sleep(1)
    c = socket.socket()
    c.connect(("localhost", 5000))

    msg = input("Enter frame: ")
    c.send(msg.encode())

    print("Server:", c.recv(1024).decode())
    c.close()
```

```
threading.Thread(target=server).start()
threading.Thread(target=client).start()
```

Output Screenshot:



The screenshot displays a Python IDE with two windows. The main window shows the code for a client-server socket program. The code defines a `server()` function that binds to `localhost` on port 5000, listens for a connection, receives a message, prints it, and sends a response. It also defines a `client()` function that connects to the server, sends a message, and prints the received response. The code uses `threading` to run both functions simultaneously.

The output window, titled "IDLE Shell 3.14.7", shows the execution of the program. It displays the following output:

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:/Users/karth/AppData/Local/Programs/Python/Python314/Client-Server Socket Program.py
Waiting for client...
Client connected
Enter frame:
Hello
Frame received: Hello

MAC Address Table:
AA:BB:CC:DD:01 -> Port 1
AA:BB:CC:DD:02 -> Port 2
Server: Frame received
```

5. StarSwitch Class:

Python Code:

```
class StarSwitch:

    def __init__(self):
        self.ports = []

    def add_port(self, host):
        self.ports.append(host)
        print(host, "connected")

    def remove_port(self, host):
        self.ports.remove(host)
        print(host, "removed")

    def broadcast(self, frame, source):
        print("\nBroadcast:", frame)
        for host in self.ports:
            if host != source:
                print("Received by", host)

# Create switch
s = StarSwitch()

# Add hosts
s.add_port("Host1")
s.add_port("Host2")
s.add_port("Host3")

# Broadcast frame
s.broadcast("Hello", "Host1")

# Remove host
s.remove_port("Host3")
```


Output Screenshot:

The screenshot displays a Python IDE window titled 'StarSwitch Class.py' with the following code:

```
class StarSwitch:
    def __init__(self):
        self.ports = []

    def add_port(self, host):
        self.ports.append(host)
        print(host, "connected")

    def remove_port(self, host):
        self.ports.remove(host)
        print(host, "removed")

    def broadcast(self, frame, source):
        print("\nbroadcast:", frame)
        for host in self.ports:
            if host != source:
                print("Received by", host)

# Create switch
s = StarSwitch()

# Add hosts
s.add_port("Host1")
s.add_port("Host2")
s.add_port("Host3")

# Broadcast frame
s.broadcast("Hello", "Host1")

# Remove host
s.remove_port("Host3")
```

The output window shows the execution results:

```
>>> - RESTART: C:/Users/karth/AppData/Local/Programs/Python/Python314/StarSwitch Cla
ss.py
Host1 connected
Host2 connected
Host3 connected

Broadcast: Hello
Received by Host2
Received by Host3
Host3 removed
>>>
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases

Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

6. Star Topology

This program accepts the number of hosts and displays the hosts connected to a central switch with Cat6 cable lengths.

Python Code:

```
# Star Topology

n = int(input("Enter number of hosts: "))

switch = "Central Switch"

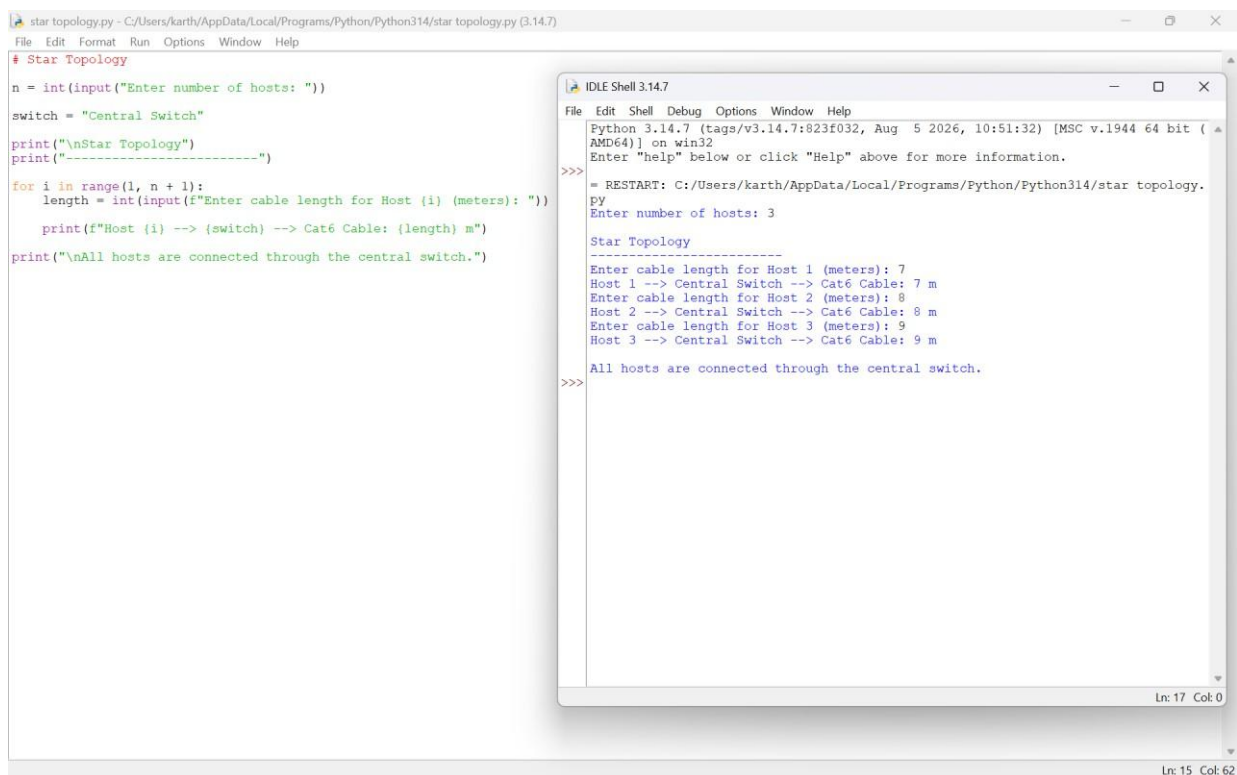
print("\nStar Topology")
print("-----")

for i in range(1, n + 1):
    length = int(input(f"Enter cable length for Host {i} (meters): "))

    print(f'Host {i} --> {switch} --> Cat6 Cable: {length} m')

print("\nAll hosts are connected through the central switch.")
```

Output Screenshot:

The screenshot shows a Python IDE window titled 'star topology.py' with the following code:

```
# Star Topology

n = int(input("Enter number of hosts: "))

switch = "Central Switch"

print("\nStar Topology")
print("-----")

for i in range(1, n + 1):
    length = int(input(f"Enter cable length for Host {i} (meters): "))
    print(f'Host {i} --> {switch} --> Cat6 Cable: {length} m')

print("\nAll hosts are connected through the central switch.")
```

Next to it is an 'IDLE Shell 3.14.7' window showing the execution output:

```
>>>
= RESTART: C:/Users/karth/AppData/Local/Programs/Python/Python314/star topology.py
Enter number of hosts: 3

Star Topology
-----
Enter cable length for Host 1 (meters): 7
Host 1 --> Central Switch --> Cat6 Cable: 7 m
Enter cable length for Host 2 (meters): 8
Host 2 --> Central Switch --> Cat6 Cable: 8 m
Enter cable length for Host 3 (meters): 9
Host 3 --> Central Switch --> Cat6 Cable: 9 m

All hosts are connected through the central switch.
>>>
```

Concept: In a star topology, every host is directly connected to a central switch.

7. UTP Cable Length Validation:

The question requires checking whether a UTP cable is within the 100-meter maximum.

Python code:

```
# UTP Cable Length Validation
```

```
def validate_segment(length_m):  
    if length_m <= 100:  
        return True, "Valid cable length"  
    else:  
        return False, "Error: Cable length exceeds 100 meters"
```

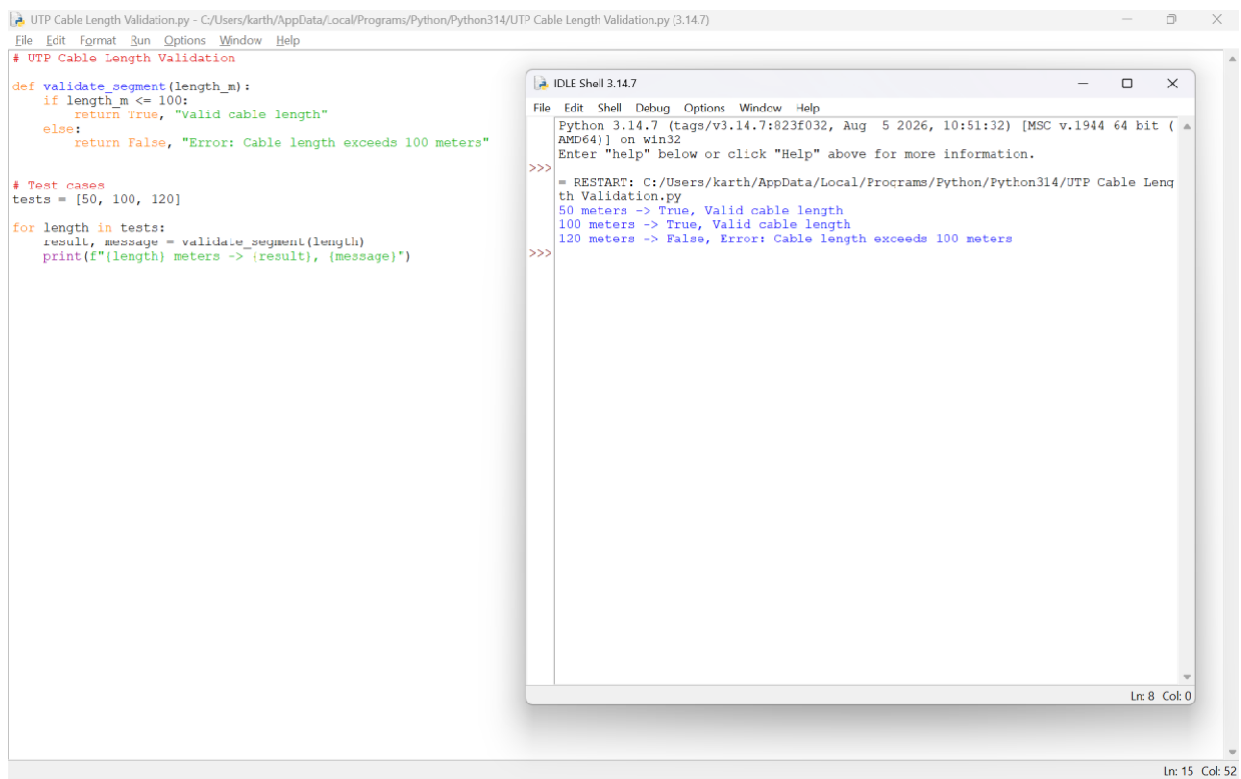
```
# Test cases
```

```
tests = [50, 100, 120]
```

```
for length in tests:
```

```
    result, message = validate_segment(length)  
    print(f'{length} meters -> {result}, {message}')
```

Output Screenshot:



Simple Interpretation:

- 50 m: Valid
- 100 m: Valid
- 120 m: Invalid because it exceeds the 100 m limit

8. CSMA/CD Simulation:

The PDF asks for four hosts, collision detection, random back-off, transmission attempts, and logging in csma_log.txt.

Python Code:

```
# Simple CSMA/CD Simulation

import random
import time

hosts = ["Host1", "Host2", "Host3", "Host4"]

log = open("csma_log.txt", "w")

for host in hosts:

    attempt = 1

    while True:
        log.write(f'{host}: Transmission attempt {attempt}\n')

        # Random collision
        collision = random.choice([True, False])

        if collision:
            log.write(f'{host}: Collision detected\n')

            backoff = random.randint(1, 3)
            log.write(f'{host}: Back-off = {backoff} seconds\n')

            print(host, "Collision! Back-off:", backoff, "seconds")
            time.sleep(backoff)

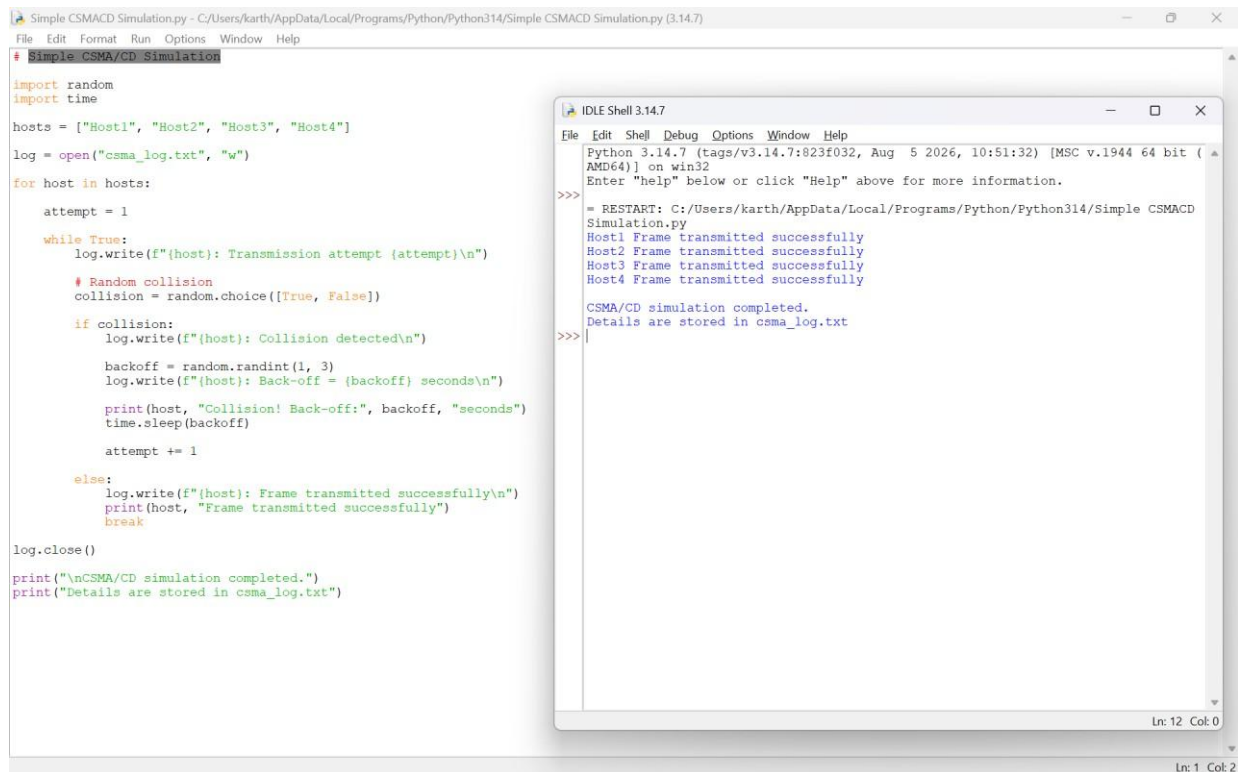
            attempt += 1

        else:
            log.write(f'{host}: Frame transmitted successfully\n')
            print(host, "Frame transmitted successfully")
            break

log.close()

print("\nCSMA/CD simulation completed.")
print("Details are stored in csma_log.txt")
```

Output Screenshot:



The screenshot shows a Python IDE window titled 'Simple CSMA/CD Simulation.py' with the following code:

```
# Simple CSMA/CD Simulation
import random
import time

hosts = ["Host1", "Host2", "Host3", "Host4"]
log = open("csma_log.txt", "w")

for host in hosts:
    attempt = 1
    while True:
        log.write(f"{host}: Transmission attempt {attempt}\n")
        # Random collision
        collision = random.choice([True, False])
        if collision:
            log.write(f"{host}: Collision detected\n")
            backoff = random.randint(1, 3)
            log.write(f"{host}: Back-off = {backoff} seconds\n")
            print(host, "Collision! Back-off:", backoff, "seconds")
            time.sleep(backoff)
            attempt += 1
        else:
            log.write(f"{host}: Frame transmitted successfully\n")
            print(host, "Frame transmitted successfully")
            break
log.close()

print("\nCSMA/CD simulation completed.")
print("Details are stored in csma_log.txt")
```

The IDE Shell window shows the output of the script:

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:/Users/karth/AppData/Local/Programs/Python/Python314/Simple CSMA/CD Simulation.py
Host1 Frame transmitted successfully
Host2 Frame transmitted successfully
Host3 Frame transmitted successfully
Host4 Frame transmitted successfully

CSMA/CD simulation completed.
Details are stored in csma_log.txt
>>>
```

The status bar at the bottom right indicates 'Ln: 12 Col: 2'.

9. Client-Server Socket Program:

Python Code:

```
import socket
import threading
import time

def server():
    s = socket.socket()
    s.bind(("localhost", 5000))
    s.listen(1)
    print("Waiting for client...")

    c, addr = s.accept()
    print("Client connected")

    msg = c.recv(1024).decode()
    print("Frame received:", msg)

    print("\nMAC Address Table:")
    print("AA:BB:CC:DD:01 -> Port 1")
    print("AA:BB:CC:DD:02 -> Port 2")
```

```
c.send("Frame received".encode())
c.close()
s.close()
```

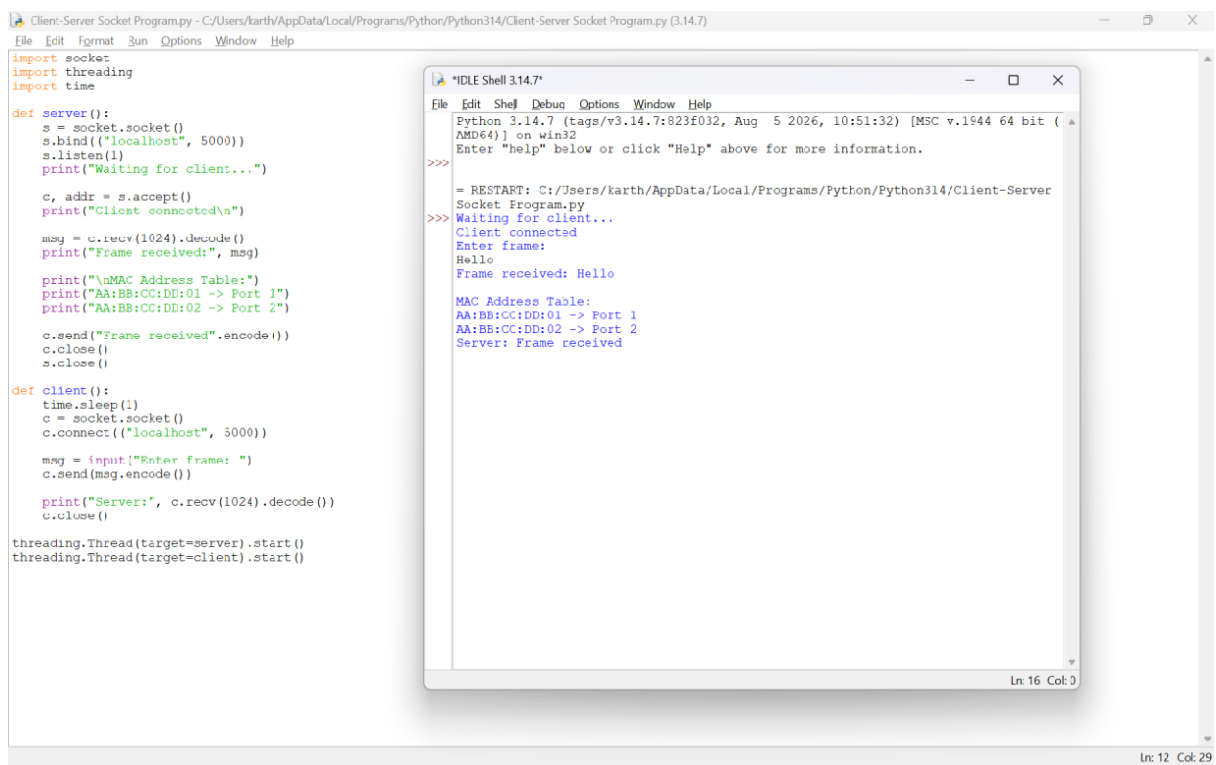
```
def client():
    time.sleep(1)
    c = socket.socket()
    c.connect(("localhost", 5000))

    msg = input("Enter frame: ")
    c.send(msg.encode())

    print("Server:", c.recv(1024).decode())
    c.close()
```

```
threading.Thread(target=server).start()
threading.Thread(target=client).start()
```

Output Screenshot:



The screenshot displays a Python IDE with two windows. The main window shows the code for a client-server socket program. The code defines a `server()` function that binds to `localhost` on port 5000, listens for a connection, receives a message, prints it, and sends a response. It also defines a `client()` function that connects to the server, sends a message, and prints the received response. The code uses `threading` to run both functions simultaneously.

The output window, titled "IDLE Shell 3.14.7", shows the execution of the program. It displays the following output:

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:/Users/karth/AppData/Local/Programs/Python/Python314/Client-Server Socket Program.py
Waiting for client...
Client connected
Enter frame:
Hello
Frame received: Hello

MAC Address Table:
AA:BB:CC:DD:01 -> Port 1
AA:BB:CC:DD:02 -> Port 2
Server: Frame received
```

The status bar at the bottom right of the IDE indicates "Ln: 12 Col: 29".

10. StarSwitch Class:

Python Code:

```
class StarSwitch:

    def __init__(self):
        self.ports = []

    def add_port(self, host):
        self.ports.append(host)
        print(host, "connected")

    def remove_port(self, host):
        self.ports.remove(host)
        print(host, "removed")

    def broadcast(self, frame, source):
        print("\nBroadcast:", frame)
        for host in self.ports:
            if host != source:
                print("Received by", host)

# Create switch
s = StarSwitch()

# Add hosts
s.add_port("Host1")
s.add_port("Host2")
s.add_port("Host3")

# Broadcast frame
s.broadcast("Hello", "Host1")

# Remove host
s.remove_port("Host3")
```

Output Screenshot:

The screenshot displays a Python IDE window titled 'StarSwitch Class.py' with the following code:

```
class StarSwitch:
    def __init__(self):
        self.ports = []

    def add_port(self, host):
        self.ports.append(host)
        print(host, "connected")

    def remove_port(self, host):
        self.ports.remove(host)
        print(host, "removed")

    def broadcast(self, frame, source):
        print("\nbroadcast:", frame)
        for host in self.ports:
            if host != source:
                print("Received by", host)

# Create switch
s = StarSwitch()

# Add hosts
s.add_port("Host1")
s.add_port("Host2")
s.add_port("Host3")

# Broadcast frame
s.broadcast("Hello", "Host1")

# Remove host
s.remove_port("Host3")
```

The IDLE Shell 3.14.7 window shows the execution output:

```
>>> - RESTART: C:/Users/karth/AppData/Local/Programs/Python/Python314/StarSwitch Cla
ss.py
Host1 connected
Host2 connected
Host3 connected

Broadcast: Hello
Received by Host2
Received by Host3
Host3 removed
>>>
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases

Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done